트리에서 LCS 쿼리 해결

구동현 . 박다솜
서울과학고등학교

On Solving LCS Queries on Trees

Donghyun Gu . Dasom Park
Seoul Science High School

국문 초록

Longest Common Subsequence(LCS) 문제는 다양하게 변형되고 응용되어 연구되는 문제이다. LCS 문제의 변형 문제인 Semi-Local LCS 문제나 Dynamic LCS 문제들은 모두 문자열들이 선형 구조를 가진다는 한계점이 있었다. 본 연구에서는 트리 구조가 가지는 범용성에 주목하여 한 문자열과 트리 위의 경로로 만들어진 문자열의 LCS를 구하는 Tree LCS 문제를 빠르게 해결하는 것이 목표이다. 이 문제를 Seaweed matrix와 Seaweed Tree, Heavy-Light Decomposition을 이용해 문자 업데이트 연산을 $O(N \log N \log M)$의 시간복잡도로, 경로 LCS 쿼리 연산을 $O(N \log N \log^2 M)$에 처리하는 Tree LCS Algorithm을 제시하며, Naïve Algorithm과 함께 사용하여 이 문제를 효율적으로 해결할 수 있을 것으로 기대한다.
주제어: LCS, Seaweed Tree, Heavy-Light Decomposition, Tree LCS Algorithm

Abstract

The Longest Common Subsequence(LCS) problem is a problem that has been transformed, applied, and studied in various ways. Both the semi-local LCS problem and the dynamic LCS problem, which are deformation problems of the LCS problem, had limitations that the strings had a linear structure. In this study, the goal is to quickly solve the Tree LCS problem of obtaining LCS of one string and a string made of paths on the tree by paying attention to the versatility of the tree structure. We present the Tree LCS Algorithm, which processes character update operations to $O(N \log N \log M)$ and path LCS query operations to $O(N \log N \log^2 M)$ time complexity using the seawed matrix, the seawed tree, and the heavy-light decomposition, and we expect to solve this problem efficiently by using it with the Naïve Algorithm.
Keywords: LCS, Seaweed Tree, Heavy-Light Decomposition, Tree LCS Algorithm

I. 서론(Introduction)

1. 필요성과 목적

두 수열의 LCS를 구하는 문제는 기본적인 컴퓨터과학 문제 중 하나이며, 두 데이터를 차이점을 찾는 data comparison 이나 생물정보학 등에 자주 응용되는 중요한 문제이다. LCS 문제의 변형 문제도 많이 연구되고 있으며, 최근에는 한 문자열과 다른 문자열의 부분 문자열, 한 문자열의 prefix와 다른 문자열의 suffix 등의 LCS를 구하는 Semi-Local LCS 문제에 대한 풀이가 등장하고 있다. [1]에서는 이 문제의 풀이들이 정적인 문자열들에 대한 풀이라는 한계에 주목하여 동적인 문자열에서 LCS에 대한 쿼리를 처리하는 새로운 자료구조인 Seaweed Tree를 만들었다.

한편, 트리 구조는 컴퓨터과학 및 다양한 시스템에서 사용하며, 많은 종류의 데이터는 트리 구조로 나타냈을 때 더 효율적인 방법으로 표현된다. 앞서 설명한 Semi-Local LCS 문제 및 Dynamic LCS 문제에서는 문자열의 형태가 선형이기에 트리 구조로의 응용이 불가능하다는 한계가 있었다. 따라서 본 연구에서는 Seaweed matrix와 Seaweed Tree, Heavy-Light Decomposition을 응용하여 트리에서 문자 업데이트와 LCS 쿼리를 효율적으로 해결하는 알고리즘인 Tree LCS Algorithm을 제시하며, LCS Algorithm과의 결합으로 제시한 문제를 빠르게 해결할 수 있을 것으로 기대한다.

2. 선행연구 및 관련이론

2.1 LCS Algorithm

LCS 문제는 길이 N, M인 문자열 A, B가 주어졌을 때 두 수열의 LCS 또는 그 길이를 구하는 문제이다. LCS 문제는 Dynamic Programming을 통해 $O(NM)$에 풀 수 있음이 알려져 있다. index가 1부터 시작하는 문자열에 대해 $dp[i][j] = (A[1,i]$와 $B[1,j]$의 LCS의 길이)로 정의했을 때 dp의 점화식은 식 (1)과 같다.[4]

$$dp[i][j] = \begin{cases} \max(dp[i-1][j], dp[i][j-1]) & (A[i] \neq B[j]) \\ dp[i-1][j-1] + 1 & (A[i] = B[j]) \end{cases} \tag{1}$$

2.2 Segment Tree

Segment Tree는 연속적인 데이터 중 특정 범위에서의 어떤 이항 연산의 수행과 데이터의 업데이트를 빠르게 처리하는 자료구조이다. 이때 데이터들은 그 이항 연산에 대한 monoid를 이루어야 한다. Segment Tree는 완전 이진 트리(Complete Binary Tree) 구조를 기반으로, Top-down 또는 Bottom-up 방식을 통하여 구현된다.
Segment Tree는 전처리, 업데이트, 쿼리의 세 가지 연산이 있다. 각각의 연산의 과정은 다음과 같다: 전처리는 Segment Tree의 리프 노드에 데이터를 모두 저장한 후, 리프 노드가 아닌 노드에 두 자식 노드의 연산 결과를 저장한다. 업데이트는 갱신할 데이터를 나타내는 리프 노드를 갱신시킨 뒤 그 노드의 조상 노드들을 순서대로 두 자식 노드들의 연산 결과로 바꾼다. 쿼리는 연산을 할 구간을 입력받아 그 구간을 표현하는 최소 개수의 노드를 찾아 연산을 수행한다. 데이터의 길이가 N일 때 Segment Tree의 공간복잡도는 $O(N)$, 깊이는 $O(\log N)$이며, 따라서 이항 연산의 시간복잡도가 $O(1)$일 때 전처리는 $O(N)$, 업데이트와 쿼리는 모두 $O(\log N)$이 걸린다.

2.3 Heavy-Light Decomposition

Heavy-Light Decomposition(HLD)는 트리를 여러 개의 chain들로 분할하여 트리의 경로에 대한 문제를 효율적으로 해결하는 알고리즘이다. HLD는 루트가 있는 트리의 각 간선을 heavy edge와 light edge로 분류한다. Heavy edge는 모든 부모 정점의 자식 정점들 중 그를 루트로 하는 서브 트리의 크기가 가장 큰 정점과 연결된 간선이며, light edge는 heavy edge가 아닌 모든 간선이다. Chain은 인접한 heavy edge들을 묶은 집합이다. 정점이 N개인 트리 상의 임의의 경로는 $O(\log N)$개의 chain으로 분할된다.[5]
HLD를 하고 난 후 트리의 각 정점 v에는 그 정점이 속한 chain의 최상위 노드 $top[v]$이 저장된다. top의 정보를 통해 두 정점 u와 v의 Lowest Common Ancestor(LCA)를 $O(\log N)$만에 찾을 수 있다. 또, 각 chain마다 chain의 정점 또는 간선의 정보를 관리하는 Segment Tree를 구성하면 Segment Tree로 해결할 수 있는 쿼리를 트리 위의 정점 u에서 정점 v로 가는 경로에서 효율적으로 해결할 수 있다.

2.4 ALCS Algorithm

All-substrings Longest Common Subsequence(ALCS) 문제는 길이 N, M인 문자열 A, B가 주어졌을 때 B의 모든 부분 문자열과 A의 LCS의 길이를 구하는 문제이다. ALCS Algorithm은 ALCS 문제를 전처리 시간복잡도 $O(NM)$, 공간복잡도 $O(N+M)$로 해결하는 자료구조를 만들어 $B[l,r]$과 A의 LCS를 구하는 쿼리가 들어왔을 때 추가 전처리 $O(N \log N)$, 쿼리당 $O(\log N)$의 시간복잡도로 online으로 해결할 수 있다.

전처리는 A와 B의 Grid Directed Acyclic Graph(GDAG)를 Fig. 1과 같이 그리는 것으로 시작한다.

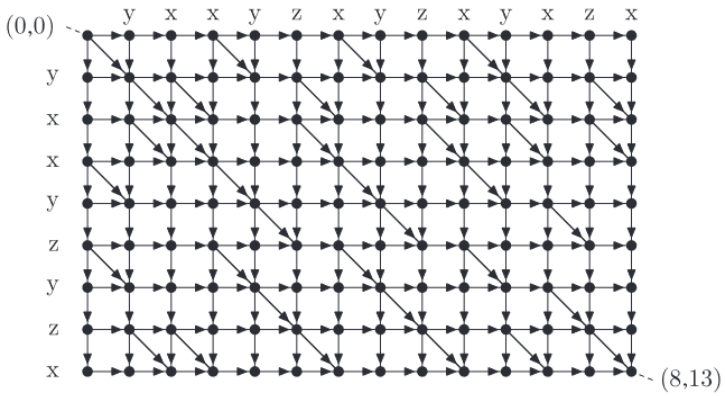


Figure 1: GDAG between two strings: A = yxxyzyzx and B = yxxyzxyzxyxzx.

GDAG는 $(N+1) \times (M+1)$개의 점이 있는 방향 그래프로, 수평 및 수직 간선의 가중치는 0이다. 대각선 간선은 index가 1부터 시작하는 문자열 A, B에 대해 $A[i] = B[j]$일 때 $(i-1, j-1)$에서 (i, j)로 가는 가중치 1인 간선들이다. GDAG의 정점 (x_1, y_1)에서 (x_2, y_2)까지 가는 최장 경로의 길이를 $C((x_1, y_1), (x_2, y_2))$로 정의했을 때 A와 $B[l, r]$의 LCS의 길이는 $C((0, l), (N, r))$이다. $i_h(i, j)$를 $C((0, y), (i, j)) = C((0, y), (i, j-1)) + 1$인 최소의 y로, $i_v(i, j)$를 $C((0, y), (i, j)) = C((0, y), (i-1, j))$인 최소의 y로 정의하면 이 값들은 GDAG의 성질에 의해 잘 정의된다. 이때 i_h와 i_v의 값들은 식 (2)와 식 (3)의 점화식으로 Dynamic Programming을 통해 구한다.

$$i_h(i, j) = \begin{cases} i_v(i, j-1) & (A[i] = B[j]) \\ \max(i_h(i-1, j), i_v(i, j-1)) & (A[i] \neq B[j]) \end{cases} \tag{2}$$

$$i_v(i, j) = \begin{cases} i_h(i-1, j) & (A[i] = B[j]) \\ \max(i_h(i-1, j), i_v(i, j-1)) & (A[i] \neq B[j]) \end{cases} \tag{3}$$

이 값들을 전처리한 후 Segment Tree나 Persistent Segment Tree를 구성하면 ALCS 쿼리 문제를 해결할 수 있다.[3]

2.5 Seaweed Matrix

식 (2)와 식 (3)의 점화식은 Fig. 2와 같이 표현할 수 있다.

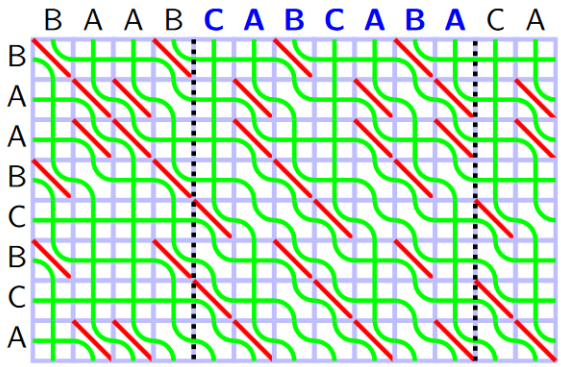


Figure 2: GDAG and seaweed braid between two strings: A = BAABCBCA and B = BAABCABCABACA.

Seaweed는 i_h와 i_v의 전이 과정을 나타낸 선이며, Seaweed braid는 Seaweed들의 집합이다. i열에서 출발한 Seaweed가

j열에 도착했다면 $i_h(N,j)=i$이며, 그런 i가 없다면 $i_h(N,j)=0$이다.
Seaweed braid는 (sub)permutation matrix P에 대응되며, 이를 Seaweed matrix라 한다. Seaweed matrix는 i열에서 출발해 j열에 도착하는 Seaweed가 존재하면 $P_{ij}=1$, 아니면 $P_{ij}=0$의 값을 가진다. Seaweed matrix간의 합성은 $\boxdot$로 표현하며, 이때 Seaweed braid는 $\boxdot$에 대해 identity matrix I를 항등원으로 가지는 monoid를 이룬다. $\boxdot$ 연산은 Divide & Conquer method로 구현했을 때 시간복잡도 $O(N\log N)$로 계산할 수 있다. A와 B의 Seaweed matrix는 $O(NM\log N)$에 구할 수 있으며, 구한 Seaweed matrix와 Segment Tree 또는 Persistent Segment Tree를 이용해서 LCS 쿼리를 빠르게 해결할 수 있다.[2]

2.6 Seaweed Tree

Seaweed Tree는 Segment Tree의 각 노드에 Seaweed matrix의 permutation 형태를 저장한 자료구조이다. Seaweed matrix의 permutation 형태는 M차원 Seaweed matrix P에 대해 각 $1\le j\le M$인 j마다 $P_{ij}=1$일 때 $\pi(j)=i$, 그런 i가 없을 때 $\pi(j)=0$인 함수 $\pi:\{1,2,\cdots,M\}\to\{0,1,2,\cdots,M\}$의 배열 형태를 뜻한다. 이후 언급되는 모든 'Seaweed matrix'는 대응되는 permutation 형태를 뜻한다.
Seaweed matrix는 $\boxdot$에 대해 monoid를 이루며, π는 monomorphism이기 때문에 이와 같은 형태의 Segment Tree는 잘 정의된다. 길이가 N, M인 문자열 A, B에 대해 Seaweed Tree의 공간복잡도는 $O(NM)$이며, 아래와 같은 연산을 할 수 있다.[1]

1. Initialize(A,B): 각 i에 대해 A와 $B[i]$의 Seaweed matrix를 구한 후 Seaweed Tree를 시간복잡도 $O(NM\log N)$으로 초기화한다.
2. Update(A,i,x): $B[i]$를 x로 변경한 후 Seaweed Tree의 노드를 $O(N\log N\log M)$으로 새로 계산한다.
3. Query(l,r): A와 $B[l,r]$의 Seaweed matrix를 시간복잡도 $O(N\log N\log M)$으로 반환한다.

II. 연구방법 및 이론(Method & Theory)

1. 도구 및 재료

모든 프로그램은 Code::Blocks 프로그램을 이용하여 C++17로 작성하였다. 프로그램 분석을 위해 사용한 사이트는 https://polygon.codeforces.com(이하 Polygon)이며, Intel(R) Core(TM) i3-8100 CPU @ 3.60GHz에서 모든 프로그램을 실행하였다.

2. 실험 과정

본 연구는 Generator 설계, Validator 설계, Checker 설계, Naïve한 알고리즘 설계, 풀이 설계, Polygon을 통한 결과 분석의 절차로 진행되었다. 본 연구가 사용하는 기호 및 해결하고자 하는 문제의 정의는 II.3.1항과 II.3.2항, 알고리즘 설계 및 증명 과정은 II.3절에서 자세히 서술한다.

2.1 Generator 설계

연구에서 구현한 프로그램의 정당성 증명 및 실행 시간 측정을 위해 Polygon에서 랜덤한 테스트 케이스를 제작해주는 프로그램을 Generator라 한다. Generator는 N, M 및 문자 업데이트의 수 Q_1, 경로 LCS 쿼리의 수 Q_2를 입력받아 알파벳 대문자로 구성된 랜덤한 문자열 A, B와 정점 M개의 랜덤한 트리 T, 그리고 $Q=Q_1+Q_2$개의 쿼리를 문제의 입력 형식에 맞게 출력한다. 본 연구에서는 랜덤 테스트 케이스를 만드는 Generator와 경로 LCS 쿼리에서 항상 트리의 지름을 이루는 두 정점이 주어지는 최악의 경우를 만드는 Generator의 두 가지 프로그램을 사용했다.
Generator를 Polygon에 업로드한 후 'Tests'탭에서 Freemarker를 통해 다수의 테스트 케이스를 제작할 수 있다.

2.2 Validator 설계

Validator는 Generator에서 만든 테스트 케이스가 올바른지 확인한다. 본 연구의 테스트 케이스는 아래와 같은 조건을 만족해야 한다.

- A, B의 길이가 각각 N, M이며, 알파벳 대문자로 이루어져 있다.
- T는 정점 M개인 트리이다.
- T가 입력된 후 Q개의 쿼리가 차례대로 입력되며, 쿼리는 모두 입력 형식에 맞아야 한다.

Validator는 생성된 테스트 케이스가 입력으로 주어졌을 때 상술된 조건에 맞지 않으면 오류를 반환한다. 완성된 Validator를 Polygon에 업로드한 후 'Validator'탭에서 선택하면 제작된 테스트 케이스를 자동으로 검사한다.

2.3 Checker 설계

Checker는 본 연구의 알고리즘이 출력한 답과 Naïve Algorithm 등이 출력한 답이 일치하는지 판별하는 프로그램이다. Checker는 Polygon에 오픈 소스로 업로드되어 있는 lcmp.cpp를 사용했다.

2.4 Naïve Algorithm 설계

시간복잡도를 고려하지 않은 가장 단순한 알고리즘을 Naïve Algorithm이라 한다. 본 연구에서 설계한 Naïve Algorithm은 아래와 같이 동작한다.

- 문자 업데이트가 입력되었을 때, $B[i]$를 x로 바꾼다.
- 경로 LCS 쿼리가 입력되었을 때, u에서 v로 가는 경로를 DFS로 탐색한 후 문자열 $B(path(u,v))$를 새로 만든다. 이 문자열과 A의 LCS의 길이는 I.2.1항에서 서술된 Dynamic Programming 기법으로 구한 후 출력한다.

이를 통해 구현된 Naïve Algorithm은 문자 업데이트를 연산당 $O(1)$, 경로 LCS 쿼리를 쿼리당 $O(NM)$의 시간복잡도로 처리한다.

2.5 결과 분석

Polygon의 'Invocations'탭에서는 Generator에서 만들어진 테스트 케이스를 실제로 프로그램에 입력하여 알고리즘의 정당성 및 실행 시간을 측정할 수 있다. 앞선 항들과 II.3절의 내용을 모두 구현한 후 Invocation을 통해 결과를 확인하고 분석했다.

3. 알고리즘 설계

3.1 기호 정의

모든 배열 또는 문자열은 index가 1부터 시작하며, 배열 또는 문자열 A의 i번째 문자를 $A[i]$로 표기한다. 배열 A와 항이 k개인 수열 $\{p_n\}$에 대해 수열 $q_i = A[p_i]$를 $A[p]$로 표기한다. 문자열 A에 대해 $A[l], A[l+1], \cdots, A[r]$을 순서대로 이어 붙인 A의 부분 문자열을 $A[l,r]$로 표기한다. 두 문자열 A, B의 LCS의 길이를 $LCS(A,B)$로 표기한다.
트리 $T=(V,E)$의 두 정점 $u, v \in V$에 대해 출발점이 u이고 도착점이 v인 T 위의 경로를 $u = p_1 \to p_2 \to \cdots \to p_k = v$라 했을 때, 수열 $\{p_n\}$을 $path(u,v)$라 한다. 또, 항이 k개인 수열 $\{a_n\}$과 문자열 S에 대해 $S[a_1], S[a_2], \cdots, S[a_k]$를 순서대로 이어 붙인 문자열을 $S(a)$로 표기한다.
List는 데이터가 순차적으로 저장되어 있는 자료구조이며, 원소를 추가할 때는 가장 뒤쪽에 추가되고, 탐색할 때는 가장 앞에 있는 원소부터 차례대로 탐색한다.

3.2 문제 정의

본 연구에서 해결하고자 하는 문제는 아래와 같다.
길이 N인 문자열 A, 길이 M인 문자열 B와, 정점의 수가 M개인 트리 T가 주어진다. T의 정점들은 각각 $1, 2, \cdots, M$의 번호가 부여되어 있으며, i번 정점에는 문자 $B[i]$가 쓰여져 있다($1 \le i \le M$). 이때 아래 두 개의 연산을 online으로 수행해야 한다.

1. 문자 업데이트: B의 i번째 문자를 주어진 문자 x로 바꾼다. 즉, i번 정점에 쓰여진 문자를 x로 바꾼다.
2. 경로 LCS 쿼리: T의 두 정점 u와 v가 주어질 때 $LCS(A, B(path(u, v)))$를 계산해서 출력한다.

II.3.3항부터 II.3.6항까지 이 문제를 해결하는 Tree LCS Algorithm(TLCS Algorithm)을 제시한다.

3.3 Heavy-Light Decomposition

트리 T의 루트를 1번 정점으로 설정한 후 HLD를 적용하여 아래와 같은 값들을 저장한다.

- dep: 각 정점의 깊이를 저장하는 배열로, $dep[v]$는 정점 v와 1번 정점 사이의 거리이다.
- par: 각 정점의 부모 정점을 저장하는 배열로, $v \neq 1$일 때 $par[v]$는 $path(v, 1)$의 두 번째 항이며, $par[1] = 0$이다.
- in: HLD를 한 트리 T에 대해 아래 조건을 만족하는 T의 chain의 정보를 저장하는 배열이다.
  - $1 \le in[v] \le M$이며, $u \neq v$일 때 $in[u] \neq in[v]$이다.
  - $par[v]$와 v가 같은 chain에 속해 있다면, $in[v] = in[par[v]] + 1$이다.
- top: Chain의 최상단 정점을 저장하는 배열로, $top[v]$는 v가 속한 chain의 dep이 가장 작은 정점의 번호이다.

이때 in 배열은 1부터 시작해서 각 정점에서 heavy edge로 연결된 자식을 첫 번째로 방문하는 DFS ordering으로 구할 수 있다. 본 연구에서 in 배열의 값은 이 방법으로 구한 값으로 고정한다.

3.4 전처리

T를 Heavy-Light Decomposition한 후 다음과 같은 과정을 통해 전처리한다.

1. $1 \le i \le M$인 각 i에 대해 $S[in[i]] = B[i]$인 길이 M의 문자열 S를 만든다.
2. S를 역순으로 쓴 문자열, 즉 $S[M], S[M-1], \cdots, S[1]$을 순서대로 이어 붙인 길이 M 문자열 R을 만든다.
3. Seaweed Tree T_S와 T_R을 만든다.
4. $T_S.\text{Initialize}(A, S)$와 $T_R.\text{Initialize}(A, R)$을 호출한다.

3.5 문자 업데이트

$B[i]$를 주어진 문자 x로 업데이트하는 과정은 다음과 같이 수행한다.

1. $S[in[i]]$와 $R[M + 1 - in[i]]$를 모두 x로 갱신한다.
2. $T_S.\text{Update}(A, in[i], x)$와 $T_R.\text{Update}(A, M + 1 - in[i], x)$를 호출한다.

3.6 경로 LCS 쿼리

주어진 두 정점 u, v에 대해 $LCS(A, B(path(u, v)))$를 구하는 과정은 아래와 같다.

1. 정수 순서쌍을 저장하는 빈 list L_f와 L_b를 만든다.
2. $top[u] \neq top[v]$인 동안 아래 과정을 반복한다.
   (a) $dep[top[u]] > dep[top[v]]$이면, L_f에 $(in[u], in[top[u]])$를 추가하고 u를 $par[top[u]]$로 바꾼다.
   (b) $dep[top[u]] \leq dep[top[v]]$이면, L_b에 $(in[top[v]], in[v])$를 추가하고 v를 $par[top[v]]$로 바꾼다.
3. $dep[u] > dep[v]$이면 L_f에, 아니면 L_b에 $(in[u], in[v])$를 추가한다.
4. L_b의 순서를 반대로 뒤집는다.
5. M차원 Seaweed matrix P를 만들고 identity permutation I로 초기화한다.
6. L_f를 탐색하며, 각 원소 (p, q)에 대해 P를 $P \boxdot T_R.\text{Query}(M + 1 - p, M + 1 - q)$로 갱신한다.
7. L_b를 탐색하며, 각 원소 (p, q)에 대해 P를 $P \boxdot T_S.\text{Query}(p, q)$로 갱신한다.
8. $P[i] = 0$인 i의 개수를 출력한다.

3.7 정당성 증명

문자 업데이트 과정에 의해서 Seaweed Tree T_S와 T_R은 갱신된 B에 대한 정보를 저장하고 있다. 즉, 문자 업데이트가 일어나지 않았을 때 II.3.6항의 알고리즘이 정확한 값을 출력한다는 것을 보이면 충분하다.
정점 u와 v의 LCA를 l이라 했을 때, 2번부터 4번까지의 과정을 통해 L_f에는 u에서 l로 가는 경로의 chain 분할들이, L_b에는 l에서 v로 가는 경로의 chain 분할들이 순서대로 저장되어 있다. 이때 L_f의 원소들은 각각 어떤 정점 x에 대해 $(in[x], in[top[x]])$로 나타낼 수 있다. $path(x, top[x])$을 $\{p_n\}$이라 했을 때 $B[p_i] = S[in[p_i]]$이며 $in[p_i] = in[p_{i+1}] + 1$이기 때문에 식 (4)가 성립한다.

$$B(p) = S(in[p]) = R(M + 1 - in[p]) = R[M + 1 - in[x], M + 1 - in[top[x]]] \tag{4}$$

L_f를 탐색하며 $B(p)$를 순서대로 이어 붙인 문자열이 $B(path(u, l))$이므로 A와 $B(p)$의 Seaweed matrix들을 순서대로 $\boxdot$ 연산을 한 Seaweed matrix가 A와 $B(path(u, l))$의 Seaweed matrix일 것이다. A와 $B(p)$의 Seaweed matrix는 식 (4)와 Seaweed Tree의 정의에 의해 $T_R.\text{Query}(M + 1 - in[x], M + 1 - in[top[x]])$와 같다. 따라서 6번까지의 과정을 마치면 P에는 A와 $B(path(u, l))$의 Seaweed matrix가 저장되어 있다.
한편, L_b의 원소들은 각각 $(in[top[y]], in[y])$로 나타낼 수 있으며, $path(top[y], y)$를 $\{q_n\}$이라 했을 때 $B[q_i] = S[in[q_i]]$이며 $in[q_{i+1}] = in[q_i] + 1$이므로 식 (5)가 성립한다.

$$B(q) = S(in[q]) = S[in[top[y]], in[y]] \tag{5}$$

식 (5)에 의해 $T_S.\text{Query}(in[top[y]], in[y])$는 A와 $B(q)$의 Seaweed matrix를 반환하므로 L_b를 탐색하며 이를 모두 합성하면 A와 $B(path(l, v))$의 Seaweed matrix가 구해진다. 따라서 7번까지의 과정을 통해 P에 A와 $B(path(u, v))$의 Seaweed matrix를 저장할 수 있다.
이제 8번 과정에서 $LCS(A, B(path(u, v)))$의 값을 구함을 보인다. A와 $B' = B(path(u, v))$의 GDAG에서 함수 C와 i_h를 정의하고, $|B'| = K$라 하자. $LCS(A, B') = C((0, 0), (N, K))$이므로 i_h의 정의 및 Seaweed matrix의 성질에 의해 식 (6)이 성립한다.

$$C((0,0),(N,K)) = \sum_{i=1}^{K} (C((0,0),(N,i)) - C((0,0),(N,i-1))) = \sum_{i=1}^{K} [i_h(N,i) = 0] = \sum_{i=1}^{N} [P[i] = 0] \tag{6}$$

위 식에 의하여 8번 과정에서는 올바른 답을 출력한다.

4. 알고리즘 분석

4.1 시간복잡도 분석

Heavy-Light Decomposition의 시간복잡도는 $O(M)$이다. 전처리 과정의 시간복잡도는 Seaweed Tree의 Initialize의 시간복잡도인 $O(NM \log N)$이다.
문자 업데이트 과정의 시간복잡도는 Seaweed Tree의 Update의 시간복잡도인 $O(N \log N \log M)$과 같다.
경로 LCS 쿼리 과정에서는 L_f와 L_b의 길이가 모두 $O(\log M)$이기 때문에 $O(\log M)$번의 Query 연산을 호출한다. 따라서 이 과정의 시간복잡도는 $O(N \log N \log^2 M)$이다.
주어지는 문자 업데이트의 개수를 Q_1, 경로 LCS 쿼리의 개수를 Q_2라 했을 때 TLCS Algorithm의 전체 시간복잡도는 $O(NM \log N + Q_1 N \log N \log M + Q_2 N \log N \log^2 M)$이다.

4.2 공간복잡도 분석

Heavy-Light Decomposition은 길이 M인 배열을 저장하기 때문에 $O(M)$의 공간을 사용한다. 전처리 과정에서는 Seaweed Tree의 공간복잡도가 $O(NM)$이며, 각 쿼리 과정에서는 동적 할당을 했을 때 추가 메모리를 $O(NM)$보다 적게 사용하므로 TLCS Algorithm의 전체 공간복잡도는 $O(NM)$이다.

III. 결과(Data & Result)

1. 랜덤 테스트 케이스

Fig. 3부터 Fig. 6까지는 랜덤한 테스트 케이스를 생성하는 Generator로 만들어진 테스트 케이스에 대해 각 경우마다 본 연구의 알고리즘과 Naïve Algorithm의 실행 시간을 5번씩 반복 측정하여 평균을 내 시각화한 그래프이다. 'Tree LCS' 범주가 본 연구의 알고리즘의 실행 시간이며, 'Naive' 범주가 Naïve Algorithm의 실행 시간을 나타낸다.
테스트 케이스를 구성할 때는 $NM = 10^6$, $Q_2 = 10Q_1$을 만족하도록 만들었고, Q_1의 값을 20, 40, 60, 80, 100으로 변화시켜가면서 구성했다.

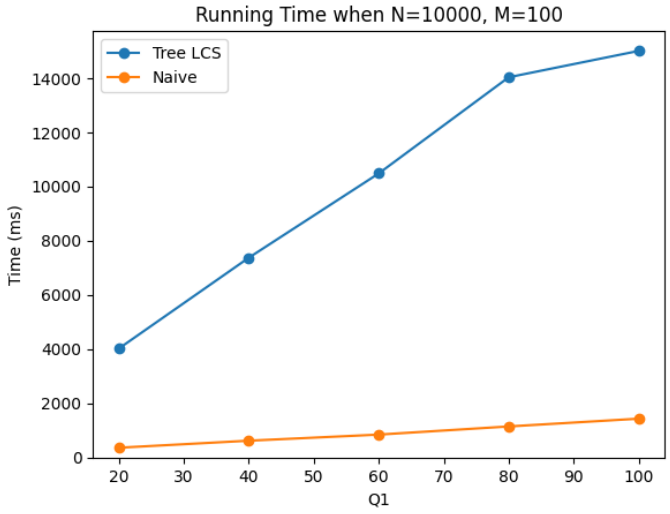


Figure 3: The runtime of algorithms when $N = 10000$, $M = 100$, $Q_2 = 10Q_1$.

$N = 10000$, $M = 100$일 때 Q_1에 따른 실행 시간을 비교한 결과 Naïve Algorithm이 TLCS Algorithm보다 항상 빠르게 나타나는 것을 확인할 수 있었다. Polygon에서 측정되는 최대 실행 시간이 15000ms이기 때문에 $Q_1 = 100$일 때 TLCS Algorithm의 실행 시간이 Q_1에 대한 선형 그래프로 나오지 않았음을 알 수 있다.

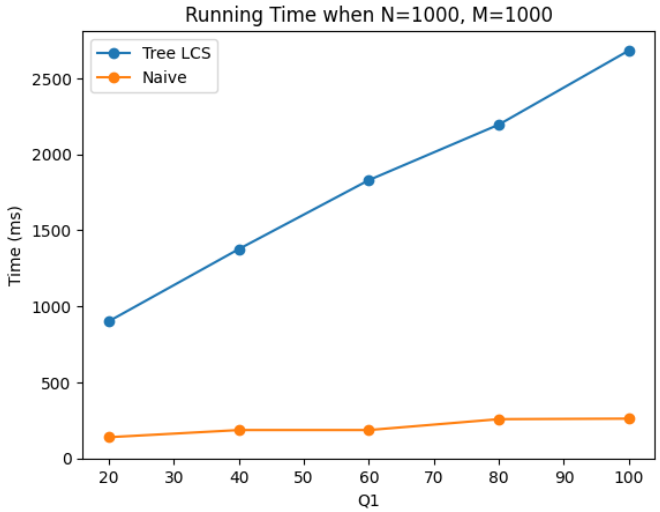


Figure 4: The runtime of algorithms when $N = 1000$, $M = 1000$, $Q_2 = 10Q_1$.

Fig. 4에서도 마찬가지로 제시한 알고리즘이 Naïve Algorithm보다 느리게 동작했다. Naïve Algorithm은 $N = 1000$, $M = 1000$인 경우에 대해서 Q_1의 변화에 따라 실행 시간이 크게 바뀌지 않았다.

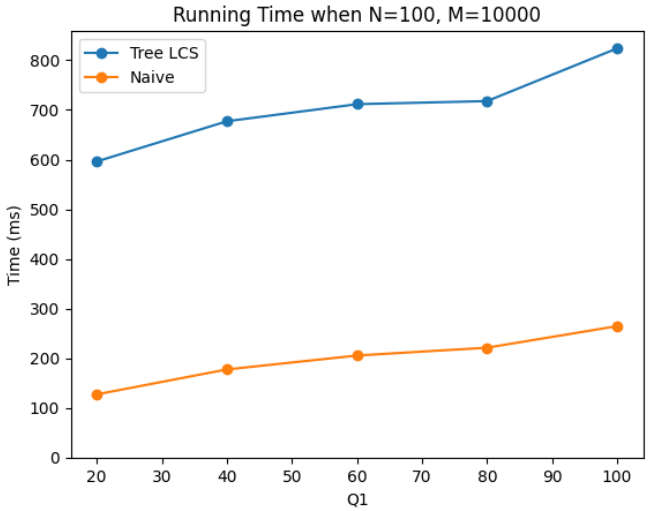


Figure 5: The runtime of algorithms when $N = 100$, $M = 10000$, $Q_2 = 10Q_1$.

$N = 100$, $M = 10000$인 경우에는 $N = 10000$인 경우나 $N = 1000$인 경우보다 두 알고리즘 간의 실행 시간 차이가 더 적게 났다. 또한 두 알고리즘의 Q_1에 따른 실행 시간의 증가율이 비슷함을 Fig. 5에서 관찰할 수 있다.

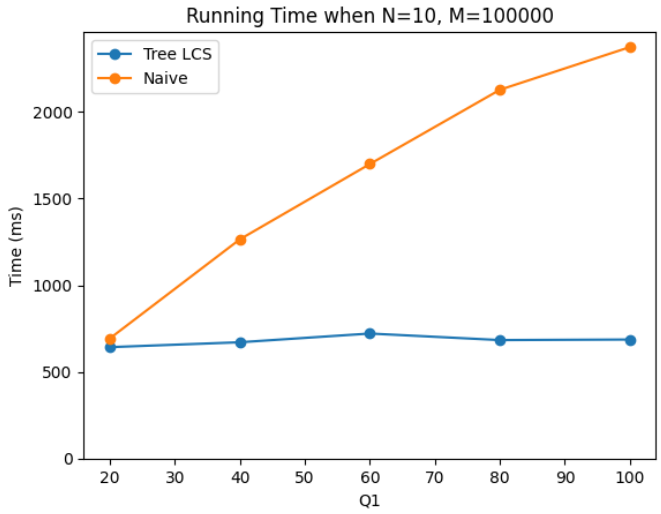


Figure 6: The runtime of algorithms when $N = 10$, $M = 100000$, $Q_2 = 10Q_1$.

Fig. 6의 경우 모든 경우에서 TLCS Algorithm이 Naïve Algorithm보다 빠르게 동작했다. Naïve Algorithm은 Q_1에 따라 실행 시간이 매우 크게 증가하는 반면, TLCS Algorithm의 실행 시간은 Q_1이 변화함에도 거의 일정했다.

2. 최악의 경우

Fig. 7부터 Fig. 10까지는 최악의 테스트 케이스를 생성하는 Generator, 즉 경로 LCS 쿼리에서 항상 트리의 지름을 이루는 두 정점이 주어지는 Generator에서 생성된 테스트 케이스에 대한 실행 시간을 나타낸 그래프이다. 마찬가지로 $NM = 10^6$일 때 N과 Q_1을 변화시켜가며 테스트 케이스를 만들었으며, 각 경우마다 5개의 테스트 케이스에 대한 평균 실행 시간을 측정했다.

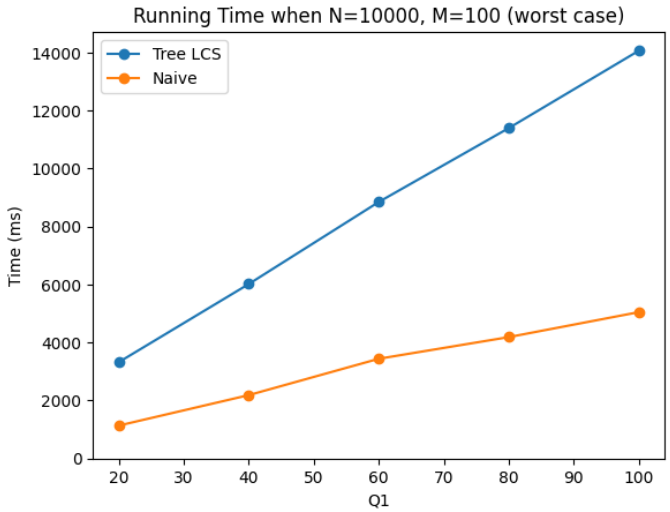


Figure 7: The runtime of algorithms in the worst case when $N = 10000$, $M = 100$, $Q_2 = 10Q_1$.

$N = 10000$일 때는 Naïve Algorithm이 TLCS Algorithm보다 빠르게 쿼리에 대한 답을 출력했다.

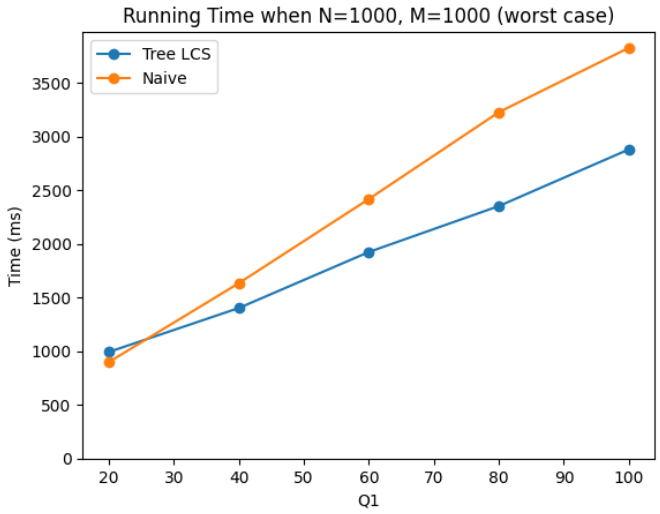


Figure 8: The runtime of algorithms in the worst case when $N = 1000$, $M = 1000$, $Q_2 = 10Q_1$.

$N = 1000$인 경우, $Q_1 = 20$일 때는 TLCS Algorithm이 Naïve Algorithm보다 느렸지만, $Q_1 \geq 40$일 때는 실행 시간이 더 적게 걸렸다. Q_1에 대한 실행 시간의 기울기 또한 Naïve Algorithm이 더 크게 나타났다.

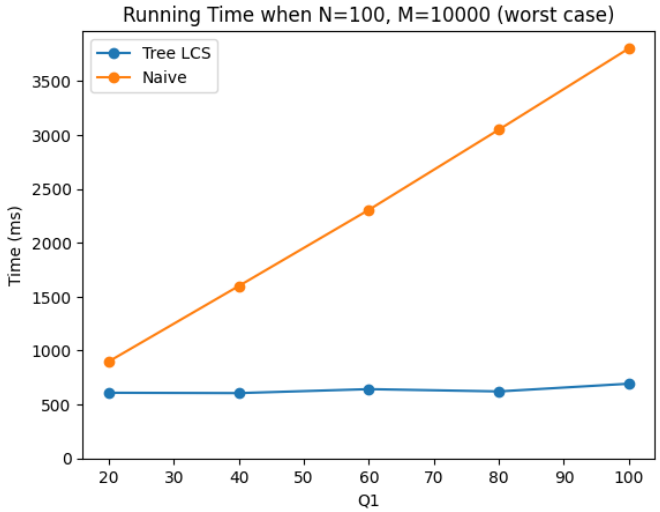


Figure 9: The runtime of algorithms in the worst case when $N = 100$, $M = 10000$, $Q_2 = 10Q_1$.

Fig. 9에서는 Naïve Algorithm이 항상 더 느렸으며 TLCS Algorithm은 Q_1에 대한 실행 시간의 증가율이 0과 비슷한 값을 가졌다. 또, Naïve Algorithm은 Fig. 7부터 Fig. 9까지 모두 실행 시간이 거의 같았다.

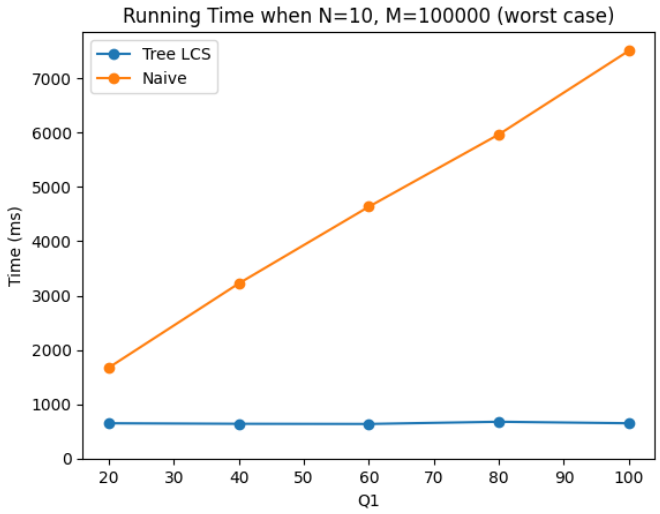


Figure 10: The runtime of algorithms in the worst case when $N = 10$, $M = 100000$, $Q_2 = 10Q_1$.

Naïve Algorithm이 모든 경우에서 제시된 알고리즘보다 느리게 작동했다. TLCS Algorithm은 Fig. 10와 Fig. 9에서 문제를 해결하는 데에 비슷한 시간이 걸렸다.

IV. 고찰(Discussion)

Naïve Algorithm의 전체 시간복잡도는 $O(N + M + Q_1 + Q_2NM)$이다. III장에서는 NM의 값을 10^6으로 일정하게 설정한 후 각 알고리즘의 Q_1에 따른 실행 시간을 측정했기 때문에 Naïve Algorithm의 실행 시간은 Q_1이 같을 때 모든 경우에서 일정해야 한다. 하지만 III.1절에서 Naïve Algorithm의 실행 시간은 모든 경우에서 크게 달랐으며, 한 경우 내에서도 반복 측정 시 실행 시간 편차가 컸다. 즉, Naïve Algorithm은 테스트 데이터에 의존적인 시간을 가지기 때문에 시간복잡도 분석을 하기 위해서는 랜덤 테스트 케이스를 통한 분석보다는 최악의 경우에 대한 실행 시간을 분석하는 것이 옳다.

최악의 경우에 $N = 10000$, $M = 100$일 때는 Naïve Algorithm이 더 빨랐다. 이는 TLCS Algorithm의 시간복잡도가 $O(NM \log N + Q_1 N \log N \log M + Q_2 N \log N \log^2 M)$이며, $\log = \log_2$일 때 $\log N \log^2 M \approx 600 > M$이기 때문에 문자 업데이트에서의 연산량과 경로 LCS 쿼리에서의 연산량이 모두 더 많아서 위와 같은 결과를 보였다.

한편, III.2절에서 나머지 경우에는 $N = 1000$, $M = 1000$, $Q_1 = 20$인 경우를 제외하고는 모두 TLCS Algorithm이 Naïve Algorithm보다 실행 시간이 작게 측정되었다. Fig. 8에서도 Q_1이 클 때는 TLCS Algorithm이 더 빠른 시간 내에 효율적으로 문제를 해결했는데, 이는 TLCS Algorithm의 전처리가 $O(NM \log N)$의 시간이 걸리기 때문이다. 실제로 TLCS Algorithm의 경우 선형 회귀를 통해 $N = 1000$, $Q_1 = 0$에서 실행 시간을 근사하면 약 500ms 정도로 전처리 과정이 없는 Naïve Algorithm이 Q_1이 작을 때 TLCS Algorhtm보다 적은 시간이 걸리게 된다. 그렇지만 Q_1에 대한 실행 시간의 변화율은 문자 업데이트 또는 경로 LCS 쿼리당 시간복잡도이기 때문에 TLCS Algorithm이 Naïve Algorithm 보다 작게 나타났으며, 따라서 Q_1이 충분히 클 때는 TLCS Algorithm의 실행 시간이 더 작게 측정되었다.

$N = 100$ 또는 $N = 10$인 경우에는 TLCS Algorithm이 매우 효율적이었으며, Q_1의 증가에 따른 실행 시간의 증가량이 거의 없었다. 예를 들어 $N = 100$, $M = 10000$일 때 $N \log N \log^2 M \approx 1.17 \times 10^5$이므로 TLCS Algorithm에서 문자 업데이트 또는 경로 LCS 쿼리를 해결하는 데에 적은 시간이 소요되고, 이는 TLCS Algorithm이 N이 작을 때 트리에서 LCS 쿼리 문제를 빠르게 해결할 수 있음을 지시한다.

V. 결론(Conclusion)

본 연구에서 제시한 알고리즘인 Tree LCS Algorithm은 제시된 트리 LCS 쿼리 문제를 전처리 $O(NM \log N)$, 문자 업데이트 $O(N \log N \log M)$, 경로 LCS 쿼리 $O(N \log N \log^2 M)$에 해결하는 알고리즘이다. 특정한 상황에서 TLCS Algorithm은 문자 업데이트를 $O(1)$에, 경로 LCS 쿼리를 $O(NM)$에 해결하는 Naïve Algorithm보다 이 문제를 효율적으로 해결할 수 있다. 구체적으로, N의 크기가 작고, Q_2의 값이 클 때 Tree LCS Algorithm은 Naïve Algorithm보다 유의미하게 빠른 시간에 문제를 해결했다. TLCS Algorithm은 M의 크기에 따른 성능의 차이는 크게 나타나지 않았는데, TLCS Algorithm의 시간복잡도가 N에 대해서는 종속적이지만, M에 대해서는 전처리를 제외한 과정에서의 시간복잡도 증가량이 매우 작기 때문이었다.
III.1절과 III.2절에서 확인한 결과에 따르면 TLCS Algorithm은 데이터에 크게 의존적이지 않은 속도를 보였다. 이는 테스트 케이스, 특히 경로 LCS 쿼리의 길이에 따른 성능의 차이가 심한 Naïve Algorithm보다 더 안정적인 효율을 가짐을 알 수 있다.

VI. 제언(Proposal)

TLCS Algorithm은 랜덤하게 만들어진 테스트 케이스에 대해서는 성능이 Naïve Algorithm보다 떨어지는 단점이 있다. 랜덤 테스트 케이스에 대한 Naïve Algorithm의 성능 향상은 랜덤하게 만들어진 트리의 지름이 너무 짧고 경로 LCS 쿼리로 입력되는 두 정점의 거리가 짧기 때문으로 추측된다. TLCS Algorithm에서 경로 LCS 쿼리를 해결할 때 u와 v 사이의 거리를 Heavy-Light Decomposition을 통해 $O(\log N)$만에 구한 후, 적절한 thresholding을 통해 TLCS Algorithm의 쿼리 처리와 Naïve Algorithm의 쿼리 처리 방법 중 더 효율적인 것을 선택하여 답을 출력한다면 TLCS Algorithm의 단점을 해결할 수 있을 것이다. 실제로 이렇게 구현할 시 경로 LCS 쿼리의 시간복잡도는 $O(N \min(M, \log N \log^2 M))$이 된다.
만들어진 Seaweed matrix를 경로 LCS 쿼리에서 l, r이 추가로 주어질 때 [1]에서 $LCS(A[l, r], B)$를 구한 방법과 비슷하게 $LCS(A[l, r], B(path(u, v)))$을 구하는 데에 사용할 수 있을 것이다. 이 방법을 통해서 같은 시간복잡도로 더 복잡한 쿼리 처리도 할 수 있다. 또한, 트리의 각 정점에 문자가 아닌 문자열이 있는 경우에도 본 연구에서 제시한 문제를 해결할 수 있을 것으로 기대된다.

References

[1] 구동현, 박상훈, 박다솜. “Seaweed Matrix를 이용한 동적인 문자열에서 LCS 문제 해결.” *서울과학고등학교* (2023)

[2] Alexander Tiskin. “Semi-local string comparison: Algorithmic techniques and applications.” *Mathematics in Computer Science* 1.4 (2008): 571-603.

[3] Alves, Carlos Eduardo Rodrigues, Edson N. Cáceres, and Siang Wun Song. “An all-substrings common subsequence algorithm.” *Discrete Applied Mathematics* 156.7 (2008): 1025-1035.

[4] Daniel S. Hirschberg. “Algorithms for the longest common subsequence problem.” *Journal of the ACM (JACM)* 24.4 (1977): 664-675.

[5] D. D. Sleator, R. E. Tarjan. “A data structure for dynamic trees.” *Journal of computer and system sciences* 26(3) (1983): 362-391.